1.4 Expressions

Variables and constants are used in ***expressions*** to define new values and to modify variables. In this section, we discuss how expressions work in Java in more detail. Expressions involve the use of ***literals***, ***variables***, and ***operators***. Since we have already discussed variables, let us briefly focus on literals and then discuss operators in some detail.

1.4.1 Literals

A ***literal*** is any "constant" value that can be used in an assignment or other expression. Java allows the following kinds of literals:

- The **null** object reference (this is the only object literal, and it is allowed to be any reference type).
- Boolean: **true** and **false**.
- Integer: The default for an integer like 176, or -52 is that it is of type **int**, which is a 32-bit integer. A long integer literal must end with an "L" or "l", for example, 176L or -52l, and defines a 64-bit integer.
- Floating Point: The default for floating-point numbers, such as 3.1415 and 135.23, is that they are **double**. To specify that a literal is a **float**, it must end with an "F" or "f". Floating-point literals in exponential notation are also allowed, such as 3.14E2 or .19e10; the base is assumed to be 10.
- Character: In Java, character constants are assumed to be taken from the Unicode alphabet. Typically, a character is defined as an individual symbol enclosed in single quotes. For example, `'a'` and `'?'` are character constants. In addition, Java defines the following special character constants:

`'\n'`	(newline)	`'\t'`	(tab)
`'\b'`	(backspace)	`'\r'`	(return)
`'\f'`	(form feed)	`'\\'`	(backslash)
`'\''`	(single quote)	`'\"'`	(double quote).

- String Literal: A string literal is a sequence of characters enclosed in double quotes, for example, the following is a string literal:

```
"dogs cannot climb trees"
```

1.4.2 Operators

Java expressions involve composing literals and variables with operators. We will survey the operators in Java in this section.

Arithmetic Operators

The following are binary arithmetic operators in Java:

$+$	addition
$-$	subtraction
$*$	multiplication
$/$	division
%	the modulo operator

This last operator, modulo, is also known as the "remainder" operator, because it is the remainder left after an integer division. We often use "mod" to denote the modulo operator, and we define it formally as

$$n \bmod m = r,$$

such that

$$n = mq + r,$$

for an integer q and $0 \leq r < m$.

Java also provides a unary minus ($-$), which can be placed in front of an arithmetic expression to invert its sign. Parentheses can be used in any expression to define the order of evaluation. Java also uses a fairly intuitive operator precedence rule to determine the order of evaluation when parentheses are not used. Unlike C++, Java does not allow operator overloading for class types.

String Concatenation

With strings, the ($+$) operator performs ***concatenation***, so that the code

```
String rug = "carpet";
String dog = "spot";
String mess = rug + dog;
String answer = mess + " will cost me " + 5 + " hours!";
```

would have the effect of making answer refer to the string

```
"carpetspot will cost me 5 hours!"
```

This example also shows how Java converts nonstring values (such as 5) into strings, when they are involved in a string concatenation operation.

Increment and Decrement Operators

Like C and C++, Java provides increment and decrement operators. Specifically, it provides the plus-one increment (++) and decrement (−−) operators. If such an operator is used in front of a variable reference, then 1 is added to (or subtracted from) the variable and its value is read into the expression. If it is used after a variable reference, then the value is first read and then the variable is incremented or decremented by 1. So, for example, the code fragment

```
int i = 8;
int j = i++;          // j becomes 8 and then i becomes 9
int k = ++i;          // i becomes 10 and then k becomes 10
int m = i--;          // m becomes 10 and then i becomes 9
int n = 9 + --i;      // i becomes 8 and then n becomes 17
```

assigns 8 to j, 10 to k, 10 to m, 17 to n, and returns *i* to value 8, as noted.

Logical Operators

Java supports the standard comparisons operators between numbers:

<	less than
<=	less than or equal to
==	equal to
!=	not equal to
>=	greater than or equal to
>	greater than

The type of the result of any of these comparison is a **boolean**. Comparisons may also be performed on **char** values, with inequalities determined according to the underlying character codes.

For reference types, it is important to know that the operators == and != are defined so that expression a == b is true if a and b both refer to the identical object (or are both **null**). Most object types support an equals method, such that a.equals(b) is true if a and b refer to what are deemed as "equivalent" instances for that class (even if not the same instance); see Section 3.5 for further discussion.

Operators defined for **boolean** values are the following:

!	not (prefix)
&&	conditional and
||	conditional or

The boolean operators && and || will not evaluate the second operand (to the right) in their expression if it is not needed to determine the value of the expression. This ***"short circuiting"*** feature is useful for constructing boolean expressions where we first test that a certain condition holds (such as an array index being valid) and then test a condition that could have otherwise generated an error condition had the prior test not succeeded.

Bitwise Operators

Java also provides the following bitwise operators for integers and booleans:

$\sim$	bitwise complement (prefix unary operator)
&	bitwise and
|	bitwise or
^	bitwise exclusive-or
<<	shift bits left, filling in with zeros
>>	shift bits right, filling in with sign bit
>>>	shift bits right, filling in with zeros

The Assignment Operator

The standard assignment operator in Java is "=". It is used to assign a value to an instance variable or local variable. Its syntax is as follows:

variable = *expression*

where *variable* refers to a variable that is allowed to be referenced by the statement block containing this expression. The value of an assignment operation is the value of the expression that was assigned. Thus, if j and k are both declared as type **int**, it is correct to have an assignment statement like the following:

```
j = k = 25;      // works because '=' operators are evaluated right-to-left
```

Compound Assignment Operators

Besides the standard assignment operator (=), Java also provides a number of other assignment operators that combine a binary operation with an assignment. These other kinds of operators are of the following form:

variable op= expression

where *op* is any binary operator. The above expression is generally equivalent to

variable = variable op expression

so that x *= 2 is equivalent to x = x * 2. However, if *variable* contains an expression (for example, an array index), the expression is evaluated only once. Thus, the code fragment

```
a[5] = 10;
j = 5;
a[j++] += 2;                    // not the same as a[j++] = a[j++] + 2
```

leaves a[5] with value 12 and j with value 6.

Operator Precedence

Operators in Java are given preferences, or precedence, that determine the order in which operations are performed when the absence of parentheses brings up evaluation ambiguities. For example, we need a way of deciding if the expression, "5+2*3," has value 21 or 11 (Java says it is 11). We show the precedence of the operators in Java (which, incidentally, is the same as in C and C++) in Table 1.3.

Operator Precedence		
	Type	**Symbols**
1	array index method call dot operator	[ ] ( ) .
2	postfix ops prefix ops cast	*exp*++ *exp*−− ++*exp* −−*exp* +*exp* −*exp* ~*exp* !*exp* (*type*) *exp*
3	mult./div.	* / %
4	add./subt.	+ −
5	shift	<< >> >>>
6	comparison	< <= > >= **instanceof**
7	equality	== !=
8	bitwise-and	&
9	bitwise-xor	^
10	bitwise-or	|
11	and	&&
12	or	||
13	conditional	*booleanExpression* ? *valueIfTrue* : *valueIfFalse*
14	assignment	= += −= *= /= %= <<= >>= >>>= &= ^= |=

Table 1.3: The Java precedence rules. Operators in Java are evaluated according to the ordering above if parentheses are not used to determine the order of evaluation. Operators on the same line are evaluated in left-to-right order (except for assignment and prefix operations, which are evaluated in right-to-left order), subject to the conditional evaluation rule for boolean && and || operations. The operations are listed from highest to lowest precedence (we use *exp* to denote an atomic or parenthesized expression). Without parenthesization, higher precedence operators are performed before lower precedence operators.

We have now discussed almost all of the operators listed in Table 1.3. A notable exception is the conditional operator, which involves evaluating a boolean expression and then taking on the appropriate value depending on whether this boolean expression is true or false. (We discuss the use of the **instanceof** operator in the next chapter.)

1.4.3 Type Conversions

Casting is an operation that allows us to change the type of a value. In essence, we can take a value of one type and ***cast*** it into an equivalent value of another type. There are two forms of casting in Java: ***explicit casting*** and ***implicit casting***.

Explicit Casting

Java supports an explicit casting syntax with the following form:

(*type*) *exp*

where *type* is the type that we would like the expression *exp* to have. This syntax may only be used to cast from one primitive type to another primitive type, or from one reference type to another reference type. We will discuss its use between primitives here, and between reference types in Section 2.5.1.

Casting from an **int** to a **double** is known as a ***widening*** cast, as the **double** type is more broad than the **int** type, and a conversion can be performed without losing information. But a cast from a **double** to an **int** is a ***narrowing*** cast; we may lose precision, as any fractional portion of the value will be truncated. For example, consider the following:

```
double d1 = 3.2;
double d2 = 3.9999;
int i1 = (int) d1;                    // i1 gets value 3
int i2 = (int) d2;                    // i2 gets value 3
double d3 = (double) i2;              // d3 gets value 3.0
```

Although explicit casting cannot directly convert a primitive type to a reference type, or vice versa, there are other means for performing such type conversions. We already discussed, as part of Section 1.3, conversions between Java's primitive types and corresponding wrapper classes (such as **int** and Integer). For convenience, those wrapper classes also provide static methods that convert between their corresponding primitive type and String values.

For example, the Integer.toString method accepts an **int** parameter and returns a String representation of that integer, while the Integer.parseInt method accepts a String as a parameter and returns the corresponding **int** value that the string represents. (If that string does not represent an integer, a NumberFormatException results.) We demonstrate their use as follows:

```
String s1 = "2014";
int i1 = Integer.parseInt(s1);        // i1 gets value 2014
int i2 = -35;
String s2 = Integer.toString(i2);     // s2 gets value "-35"
```

Similar methods are supported by other wrapper types, such as Double.

Implicit Casting

There are cases where Java will perform an ***implicit cast*** based upon the context of an expression. For example, you can perform a *widening* cast between primitive types (such as from an **int** to a **double**), without explicit use of the casting operator. However, if attempting to do an implicit *narrowing* cast, a compiler error results. For example, the following demonstrates both a legal and an illegal implicit cast via assignment statements:

```
int i1 = 42;
double d1 = i1;          // d1 gets value 42.0
i1 = d1;                 // compile error: possible loss of precision
```

Implicit casting also occurs when performing arithmetic operations involving a mixture of numeric types. Most notably, when performing an operation with an integer type as one operand and a floating-point type as the other operand, the integer value is implicitly converted to a floating-point type before the operation is performed. For example, the expression 3 + 5.7 is implicitly converted to 3.0 + 5.7 before computing the resulting **double** value of 8.7.

It is common to combine an explicit cast and an implicit cast to perform a floating-point division on two integer operands. The expression (**double**) 7 / 4 produces the result 1.75, because operator precedence dictates that the cast happens first, as ((**double**) 7) / 4, and thus 7.0 / 4, which implicitly becomes 7.0 / 4.0. Note however that the expression, (**double**) (7 / 4) produces the result 1.0.

Incidentally, there is one situation in Java when only implicit casting is allowed, and that is in string concatenation. Any time a string is concatenated with any object or base type, that object or base type is automatically converted to a string. Explicit casting of an object or base type to a string is not allowed, however. Thus, the following assignments are incorrect:

```
String s = 22;                          // this is wrong!
String t = (String) 4.5;                // this is wrong!
String u = "Value = " + (String) 13;    // this is wrong!
```

To perform a conversion to a string, we must use the appropriate toString method or perform an implicit cast via the concatenation operation. Thus, the following statements are correct:

```
String s = Integer.toString(22);        // this is good
String t = "" + 4.5;                    // correct, but poor style
String u = "Value = " + 13;             // this is good
```